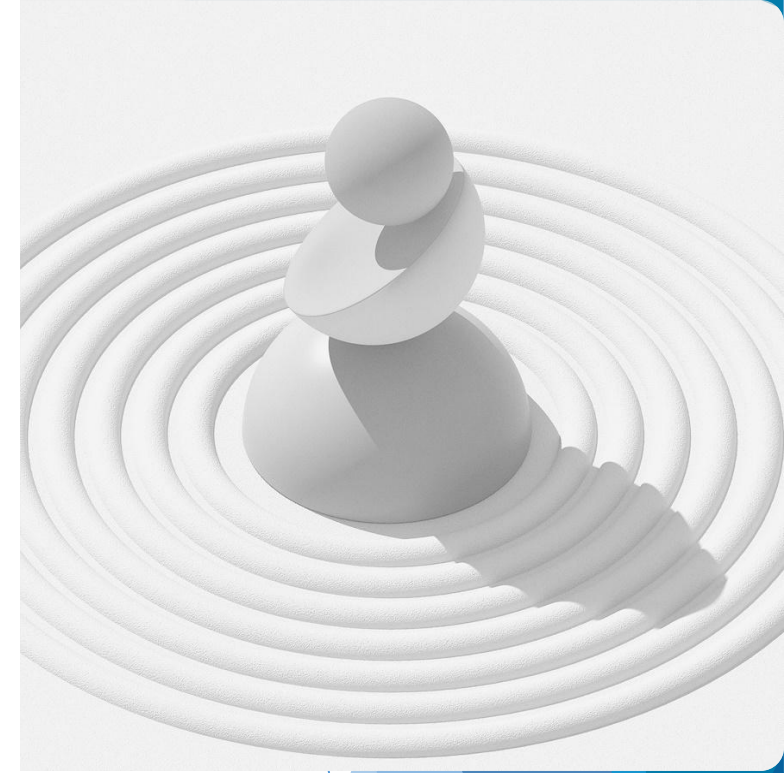


Presented by Nirvana Popescu

# Lesson 6: Software Architecture

# Outline

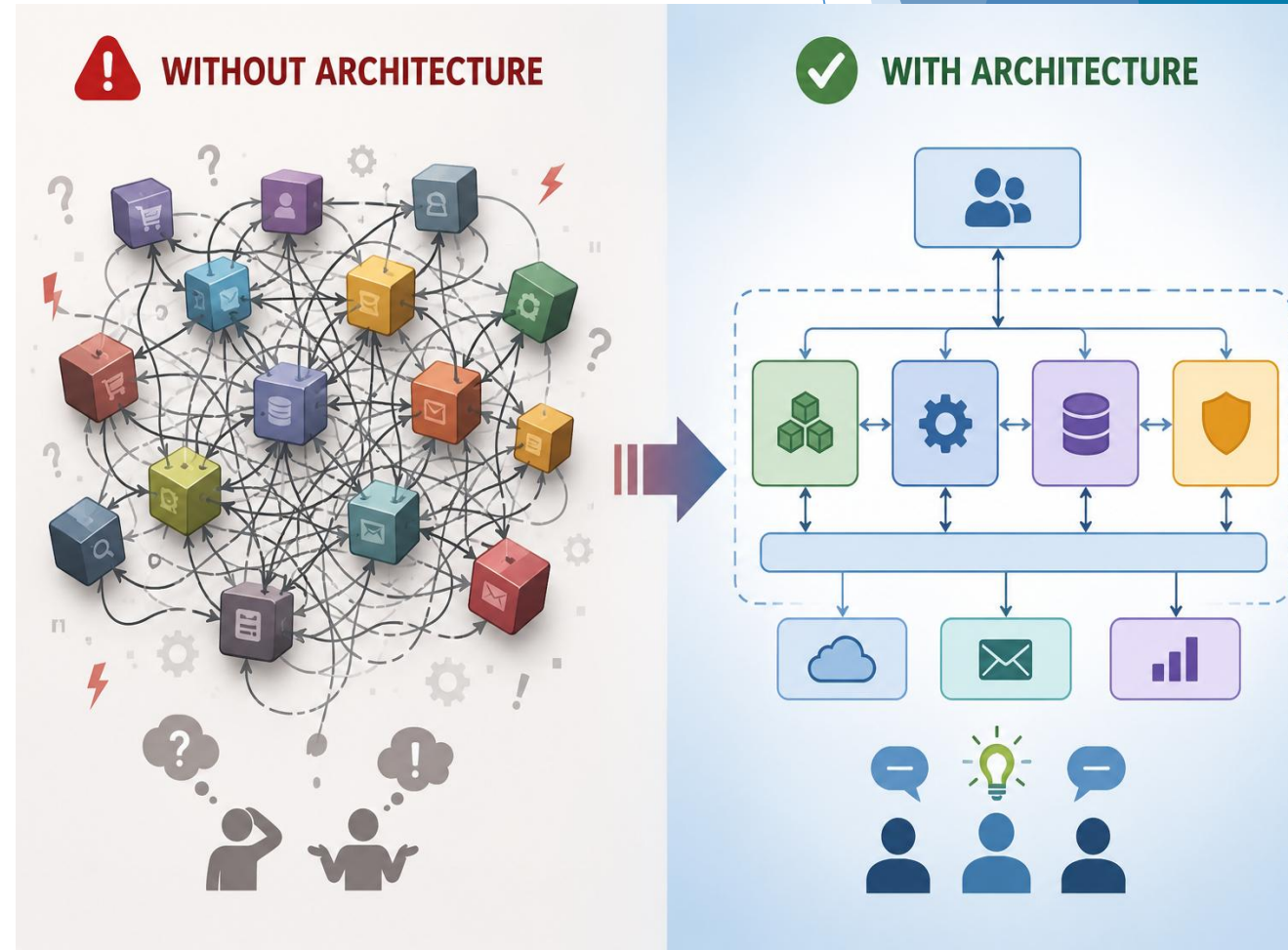
- ▶ From Requirements to System Structure
- ▶ Why Do We Need Software Architecture?
- ▶ What is Software Architecture?
- ▶ Architecture vs Design
- ▶ Main Elements of Embedded Software Architecture
- ▶ Quality Attributes Drive Architecture
- ▶ Requirements-Driven Architecture
- ▶ Mapping Requirements to Architecture
- ▶ Benefits of Requirements-Driven Architecture





# Why Do We Need Software Architecture?

- ▶ A growing software system becomes difficult to manage without structure.
- ▶ Without architecture:
  - ▶ Components become tightly coupled
  - ▶ Changes become expensive
  - ▶ Reuse is limited
  - ▶ Testing becomes harder
  - ▶ Quality attributes are difficult to achieve
- ▶ With architecture:
  - ▶ organized structure
  - ▶ clear responsibilities
  - ▶ easier evolution
  - ▶ improved communication
  - ▶ better system quality



# What is Software Architecture?

## ► Definition

*Software architecture is the fundamental organization of a software system, consisting of its components, their relationships, and the principles guiding their design and evolution.*

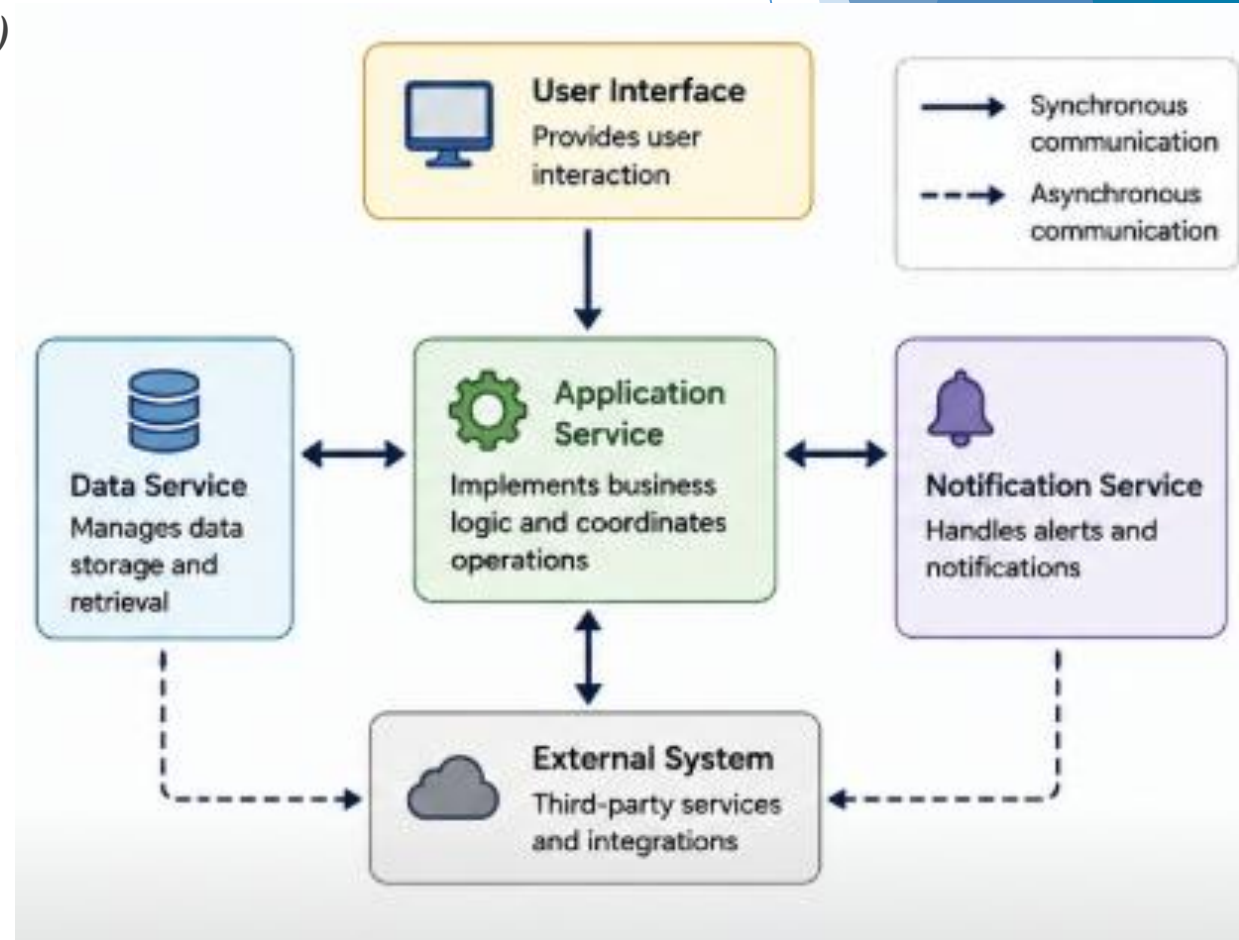
(ISO/IEC/IEEE 42010)

## ► Architecture describes:

- major components
- interfaces
- communication
- responsibilities
- constraints
- design decisions

## ► Architecture answers:

- What are the major building blocks?
- How do they communicate?
- How is complexity managed?



# ARCHITECTURE

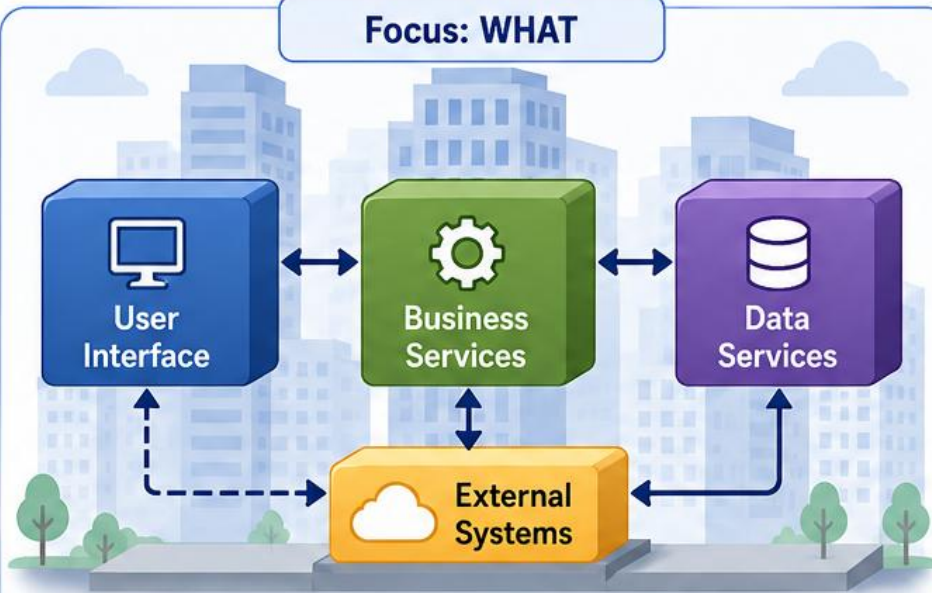
The big picture – structure & organization

VS

# DESIGN

The details – how things are built

## Focus: WHAT



Defines major components and their responsibilities (subsystems, modules, services)



Defines interfaces and communication between components



Addresses quality attributes (performance, security, scalability, reliability)



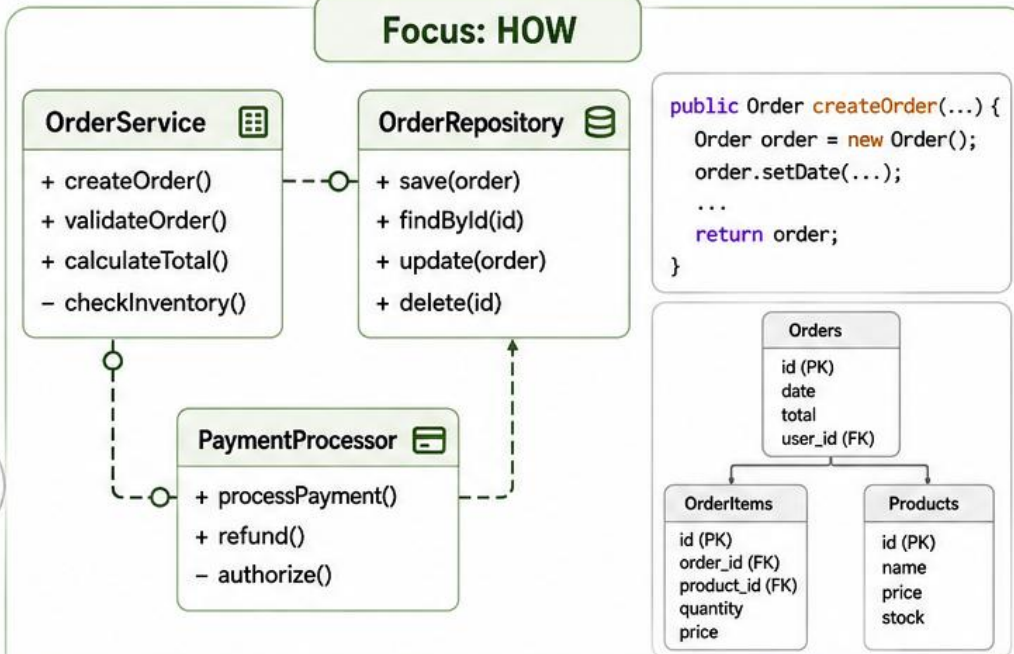
Guided by principles and constraints



Changes are expensive – impacts the whole system

VS

## Focus: HOW



Defines components at a detailed level (classes, methods, data structures)



Defines algorithms, data models, UI details



Focuses on implementation decisions



Changes are cheaper – localized impact



Answers: How does each part work?



In short:



## ARCHITECTURE

Decides the structure of the system



## DESIGN

Decides the details within that structure



Good architecture enables good design.

# Main Elements of the Embedded Software Architecture

- A software architecture typically defines:

## Connectors

- Function Calls
- Inter-Task Communication
- Message Queues
- Shared Memory
- Publish/Subscribe Events
- Communication Buses (CAN, LIN, Ethernet, SPI, I2C)

## Components

- Application Functions
- Control Algorithms
- Device Drivers
- Middleware / Services
- Hardware Abstraction Layer
- Diagnostics & Monitoring

## Interfaces

- Provided Services
- Required Services
- Hardware Interfaces
- Communication Interfaces
- Sensor & Actuator Interfaces

## Constraints

- Real-Time Performance
- Memory Footprint
- CPU Utilization
- Safety (IEC 61508, ISO 26262, etc.)
- Security
- Reliability & Availability
- Power Consumption

# Quality Attributes Drive Architecture

- ▶ Architecture is not only about functionality.  
It must satisfy quality attributes.

- ▶ Examples:

- ▶ Performance
- ▶ Scalability
- ▶ Reliability
- ▶ Availability
- ▶ Maintainability
- ▶ Safety
- ▶ Security
- ▶ Portability

- ▶ These attributes strongly influence architectural decisions.
- ▶ Many architectural decisions are made not because of functionality, but because of quality requirements.

**Requirement:**

"The system shall respond within 100 ms."

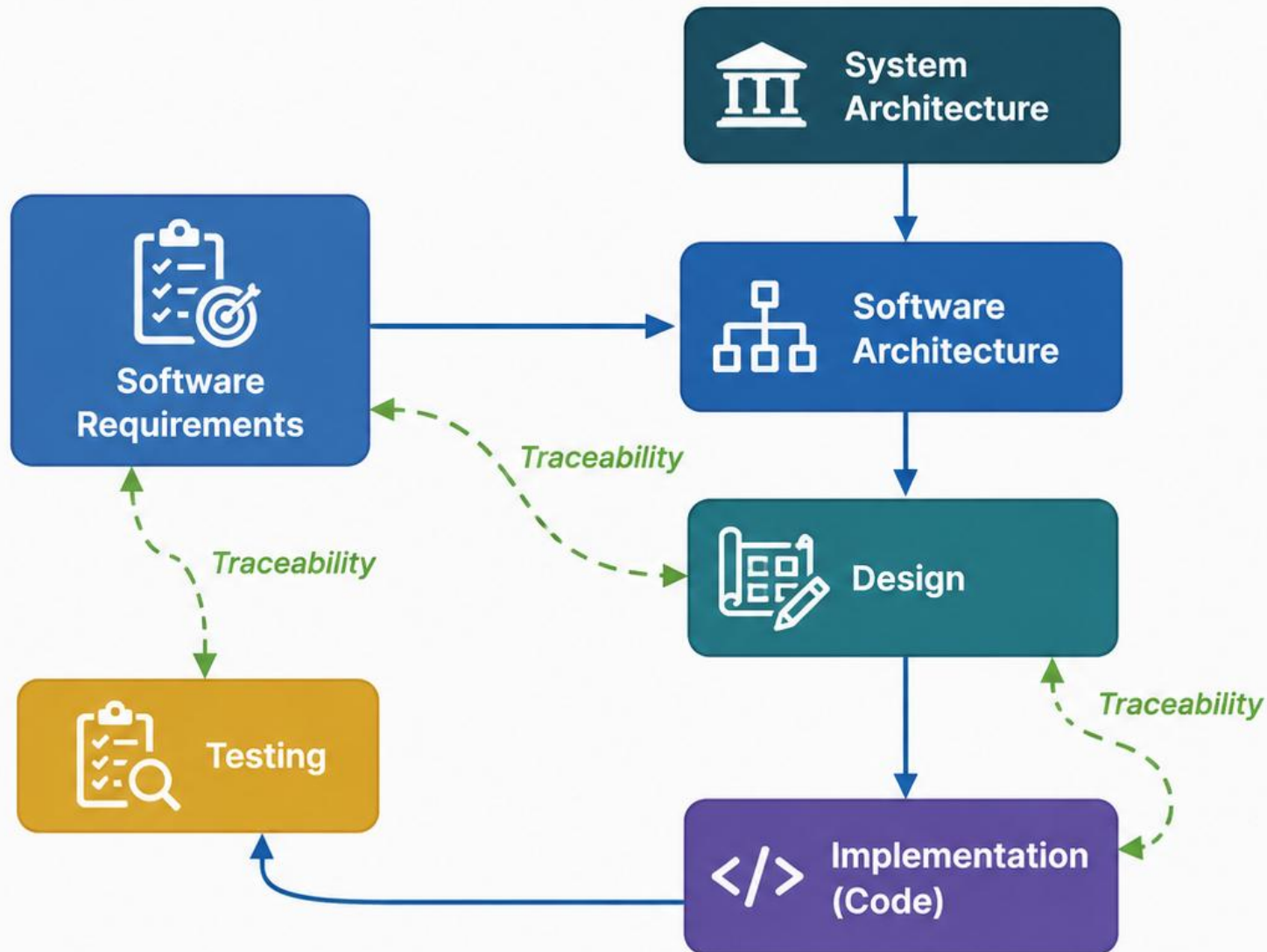


**Architecture may require:**

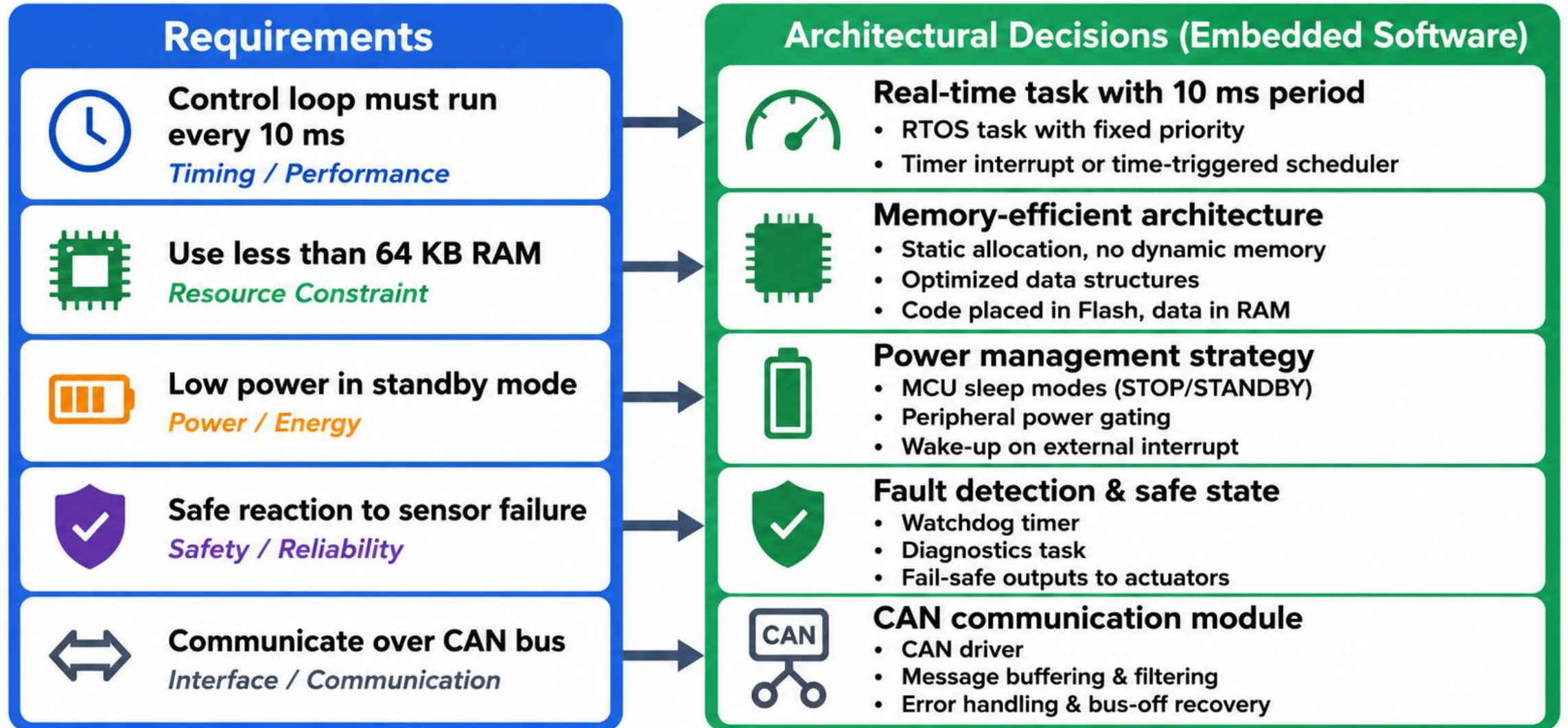
caching  
parallel processing  
distributed services

# Requirements-Driven Architecture

Software architecture should be *driven by requirements*



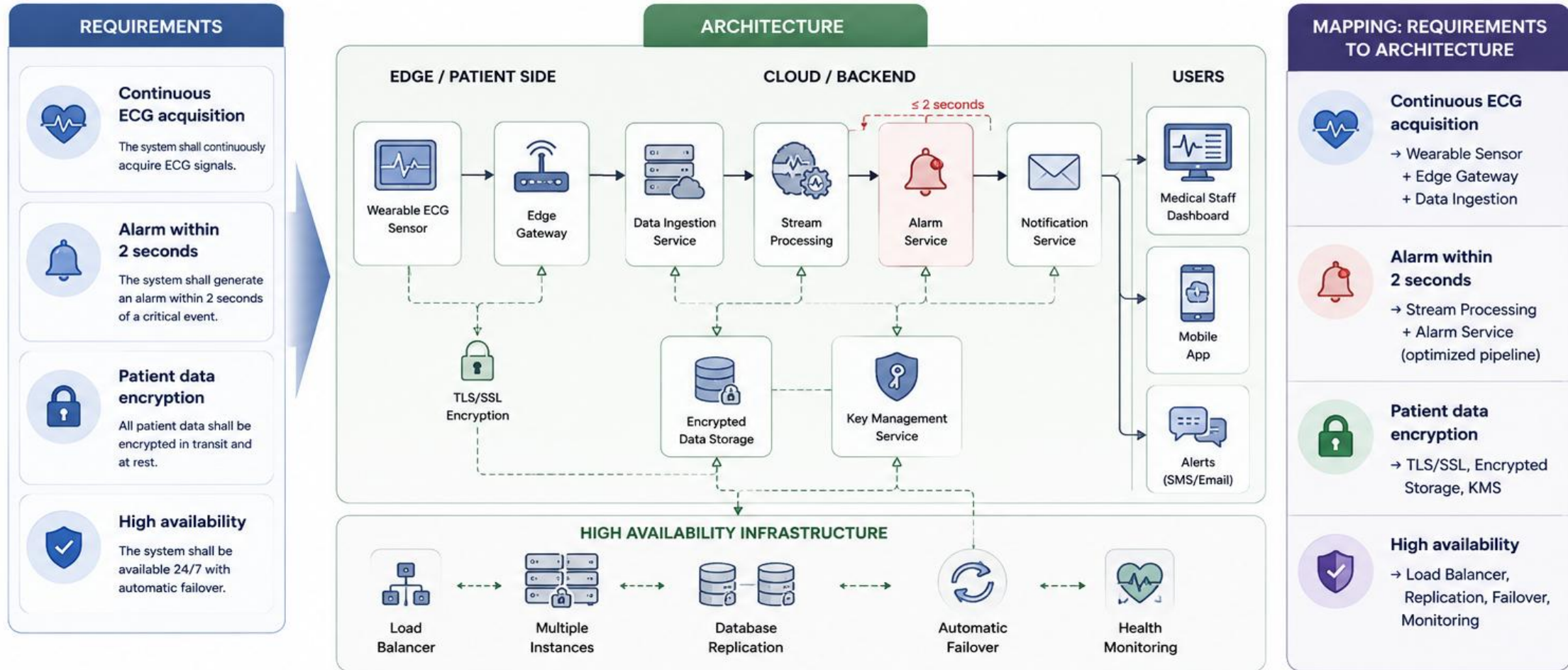
# Mapping Requirements to Architecture



**Requirements drive architectural choices in embedded systems.**

# Example: Requirements-Driven Architecture

Suppose we develop an autonomous medical monitoring system.



## Key Idea

Requirements drive architectural decisions. Each requirement is satisfied by specific architectural components and mechanisms.



**Requirements**  
(What we need)



**Architecture**  
(How we organize)



**Implementation**  
(Build the system)



**Validation**  
(Verify & Monitor)

# Benefits of Requirements-Driven Architecture

- ▶ Better traceability
- ▶ Easier validation
- ▶ Better maintainability
- ▶ More predictable quality
- ▶ Reduced redesign effort
- ▶ Easier certification (important in safety-critical systems)



Good architecture is not built around technology—it is built around requirements.

# Summary

- ▶ **Software architecture is the blueprint of a system** - It defines the system's high-level structure, major components, interfaces, and interactions before implementation begins.
- ▶ **Requirements drive architectural decisions** - Functional requirements determine **what** the system must do, while non-functional requirements determine **how well** it must do it.
- ▶ **Architecture balances quality attributes** - Architectural decisions are largely driven by performance, security, reliability, scalability, maintainability, and other quality requirements.
- ▶ **Traceability links requirements to implementation** - Every major architectural decision should be justified by one or more requirements, enabling easier validation, maintenance, and change management.
- ▶ **Good architecture enables successful software systems** - A well-designed architecture reduces complexity, supports evolution, improves communication among stakeholders, and provides a solid foundation for design, implementation, testing, and future maintenance.

*A good software architecture is not defined by the technologies it uses, but by how effectively it transforms requirements into a maintainable, reliable, and high-quality software system.*

